

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



MẠNG MÁY TÍNH

Bài tập lớn

Lớp L07

Học kỳ 2, năm học 2025–2026

Giảng viên hướng dẫn Bùi Xuân Giang
Lớp L07

Danh sách thành viên nhóm

STT	Họ và tên	MSSV	Đóng góp
1	Nguyễn Thái Hoàng	2311062	100%
2			
3			
4			
5			

TP. Hồ Chí Minh, tháng 05 năm 2026

Mục lục

1	Hệ thống chat lai HTTP và P2P	1
1.1	Architecture Overview	1
1.1.1	Thành phần chính	1
1.1.2	Control plane và data plane	1
1.2	Non-blocking I/O Mechanism	2
1.2.1	Blocking và non-blocking socket	2
1.2.2	Selector event loop	2
1.2.3	Partial send/recv và frame buffer	2
1.2.4	Gắn với PeerManager	3
1.3	HTTP Server & Authentication	3
1.3.1	HTTP authentication	3
1.3.2	Luồng xác thực hoàn chỉnh	4
1.3.3	Cookie-based session	6
1.3.4	Xử lý lỗi và phản hồi xác thực	7
1.3.5	Nhận xét về lựa chọn thiết kế	7
1.4	Client-server Bootstrap	8
1.4.1	Peer registration	8
1.4.2	Heartbeat	8
1.4.3	Discovery	9
1.5	P2P Protocol Design	9
1.5.1	Frame format	9
1.5.2	Message types	10
1.5.3	Handshake	10
1.5.4	Connection state machine	10
1.5.5	Reliability trong phạm vi ứng dụng	11
1.6	Channel & DHT Extension	11
1.6.1	Hash channel name thành key	11
1.6.2	Route qua ring	11
1.6.3	Owner fan-out đến members	12
1.6.4	Synchronization	12

1.7	Testing & Failure Handling	12
1.7.1	Test cases	12
1.7.2	Failure handling	13
1.7.3	Kết luận kiểm thử	13

1. Hệ thống chat lai HTTP và P2P

1.1 Architecture Overview

Hệ thống được thiết kế theo mô hình lai: HTTP được dùng cho phần điều khiển, còn TCP Peer-to-Peer được dùng cho dữ liệu tin nhắn trực tiếp giữa các peer. Mục tiêu của kiến trúc là tách rõ *control plane* và *data plane* để tracker chỉ hỗ trợ khám phá peer, không trở thành nút trung chuyển mọi tin nhắn.

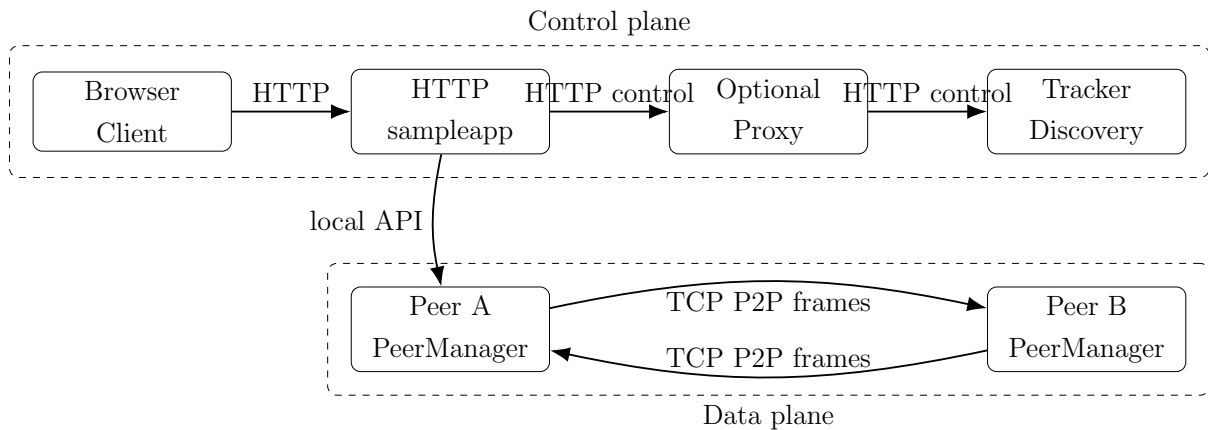
1.1.1 Thành phần chính

- **Browser/Client:** giao diện người dùng, gửi request HTTP đến sampleapp.
- **HTTP sampleapp:** phục vụ UI, xử lý request từ browser, quản lý PeerManager cục bộ và gọi tracker để bootstrap.
- **Tracker:** lưu trạng thái peer đang hoạt động, hỗ trợ register, heartbeat và discovery.
- **PeerManager:** quản lý listener TCP, kết nối P2P, hàng đợi gửi và bộ đệm nhận frame.
- **Proxy tùy chọn:** có thể đứng giữa sampleapp và tracker để mô phỏng gateway hoặc reverse proxy.

1.1.2 Control plane và data plane

Control plane gồm các request HTTP từ sampleapp đến tracker: đăng ký peer, heartbeat, lấy danh sách peer và lấy thông tin kết nối. Các thông điệp này nhỏ, có tính điều phối và không chứa luồng chat trực tiếp.

Data plane là kết nối TCP P2P giữa các peer. Khi peer A đã biết địa chỉ của peer B thông qua tracker, peer A mở kết nối TCP trực tiếp đến peer B và gửi frame JSON theo protocol riêng của hệ thống. Tin nhắn live không đi qua tracker.



Hình 1.1: Kiến trúc tổng quan và phân tách control plane/data plane

1.2 Non-blocking I/O Mechanism

1.2.1 Blocking và non-blocking socket

Với blocking socket, lời gọi `accept()`, `recv()` hoặc `send()` có thể giữ luồng thực thi cho đến khi có kết nối, có dữ liệu hoặc buffer hệ điều hành sẵn sàng. Cách này đơn giản nhưng dễ làm một peer bị treo nếu một kết nối chậm hoặc không phản hồi.

Với non-blocking socket, socket trả về ngay cả khi thao tác chưa thể hoàn tất. Chương trình không chờ bị động trên từng socket, mà dùng selector để biết socket nào đã sẵn sàng đọc hoặc ghi. Nhờ vậy một event loop có thể quản lý nhiều kết nối đồng thời mà không cần tạo một thread cho mỗi peer.

1.2.2 Selector event loop

Selector theo dõi tập socket đã đăng ký và báo readiness:

- **Read readiness:** listener sẵn sàng `accept()`, hoặc socket kết nối đã có byte để đọc.
- **Write readiness:** socket có thể gửi thêm byte từ outbound queue.
- **Timeout:** event loop có thể dùng timeout để kiểm tra handshake deadline, ping interval và cleanup peer lỗi.

Readiness không có nghĩa là toàn bộ message đã có sẵn hoặc toàn bộ payload có thể gửi một lần. Vì vậy implementation phải xử lý partial `recv()` và partial `send()`.

1.2.3 Partial send/recv và frame buffer

TCP là byte stream, không giữ ranh giới message. Một lần `recv()` có thể nhận nửa frame, đúng một frame hoặc nhiều frame liền nhau. Do đó mỗi connection cần một receive buffer:

1. đọc byte mới và append vào buffer;
2. nếu buffer chưa đủ 4 byte length prefix thì chờ thêm;
3. đọc độ dài frame từ 4 byte đầu;
4. nếu buffer chưa đủ payload JSON thì chờ thêm;
5. khi đủ frame, parse JSON và dispatch message type;
6. giữ lại byte dư cho frame kế tiếp.

Tương tự, `send()` có thể chỉ gửi được một phần payload. Mỗi connection cần outbound queue chứa các buffer chưa gửi xong. Khi selector báo writable, PeerManager tiếp tục gửi phần còn lại.

1.2.4 Gắn với PeerManager

PeerManager dùng non-blocking I/O theo các nguyên tắc:

- listener socket được đặt non-blocking và đăng ký read event;
- socket sau `accept()` cũng được đặt non-blocking;
- mỗi connection có frame buffer riêng cho inbound data;
- mỗi connection có outbound queue riêng cho pending frames;
- event loop xử lý read/write readiness, handshake timeout, heartbeat và cleanup;
- mọi message gửi ra đều được encode thành frame trước khi đưa vào queue.

1.3 HTTP Server & Authentication

1.3.1 HTTP authentication

HTTP authentication chuẩn hóa cách server yêu cầu client chứng minh danh tính. Theo RFC 7235, khi client truy cập tài nguyên cần xác thực nhưng chưa có credential hợp lệ, server trả về mã 401 `Unauthorized` kèm header `WWW-Authenticate`. Header này cho biết authentication scheme và realm mà client phải dùng.

Client gửi credential trong header `Authorization`. Hai scheme cổ điển được mô tả trong RFC 2617 là:

- **Basic:** username và password được nối bằng dấu hai chấm rồi Base64 encode. Scheme này đơn giản nhưng phải đi kèm HTTPS nếu dùng trong môi trường thật.

- **Digest:** client không gửi password trực tiếp mà gửi digest tính từ username, password, nonce và thông tin request. Digest giảm rủi ro lộ password so với Basic nhưng phức tạp hơn.

Trong hệ thống đã triển khai, nhóm chọn mô hình lai: Basic/Digest dùng cho bước xác thực ban đầu hoặc cho các route bảo vệ khi client chưa có session; sau khi đăng nhập thành công, server phát session cookie để các request tiếp theo không phải gửi lại username/password. Cách làm này phù hợp với yêu cầu đề bài vì vừa bám theo khung xác thực HTTP chuẩn, vừa tận dụng cơ chế cookie để duy trì trạng thái đăng nhập.

Credential người dùng được nạp từ `db/users.json` và cache trong bộ nhớ để tra cứu nhanh. Sau khi kiểm tra hợp lệ, server tạo một session id ngẫu nhiên, lưu ánh xạ session với user trong bộ nhớ kèm thời gian sống, rồi trả cookie phiên cho browser. Ngoài ra, implementation còn lưu trạng thái user đang hoạt động theo từng HTTP port để tránh cùng một tài khoản đăng nhập đồng thời ở nhiều instance backend khác nhau.

1.3.2 Luồng xác thực hoàn chỉnh

Hình dung luồng xử lý end-to-end của hệ thống như sau:

Listing 1.1: Luồng xác thực đầy đủ của HTTP server

Client	HTTP Server / Middleware	Auth store
POST /login + JSON creds		
----->		
	parse method, headers, body	
	----->	
	validate username/password	
	<-----	
	create session + set cookie	
<-----		
200 OK + Set-Cookie		
GET /api/self + Cookie		
----->		
	verify session	
	----->	
	<-----	
<-----		
200 OK + protected resource		
POST /logout + Cookie		
----->		

```

|                                     | delete session + expire cookie |
|<-----|                                     |
| 200 OK + Set-Cookie Max-Age=0 |
|                                     |
| GET /api/self (không có auth) |
|----->|                                     |
|<-----|                                     |
| 401 + WWW-Authenticate         |

```

Luồng trên có thể diễn giải thành các bước cụ thể:

1. Client gửi request POST /login kèm body JSON chứa **username** và **password**.
2. Lớp request parser tách request line, header, body và cookie; nếu **Content-Type** là **application/json** thì body được parse sang object để handler đăng nhập sử dụng.
3. Handler /login ưu tiên kiểm tra credential trong JSON body. Nếu body không chứa đủ thông tin, server có thể fallback sang **Authorization: Basic** hoặc **Authorization: Digest**.
4. Nếu credential hợp lệ, server tạo session mới, gắn cookie phiên vào response thông qua header **Set-Cookie**.
5. Ở các request sau, client chỉ cần gửi lại cookie phiên; middleware sẽ kiểm tra session trước khi cho phép đi vào protected route.
6. Khi người dùng chọn logout, client gửi POST /logout kèm cookie phiên hiện tại; server xóa session phía server và trả lại **Set-Cookie** với **Max-Age=0** để buộc trình duyệt hủy cookie.
7. Sau logout, nếu client truy cập lại route bảo vệ mà không xác thực mới, server trả về 401 Unauthorized và gắn **WWW-Authenticate** để yêu cầu đăng nhập lại.
8. Nếu client dùng Basic/Digest hợp lệ tại protected route, request vẫn có thể được chấp nhận ngay cả khi chưa có session cookie.

Ví dụ request đăng nhập bằng JSON:

Listing 1.2: Request login với JSON credential

```

POST /login HTTP/1.1
Host: 127.0.0.1:2026
Content-Type: application/json

{"username": "admin", "password": "xoilactv"}

```


1.3.3 Cookie-based session

RFC 6265 định nghĩa cơ chế HTTP cookie. Server gửi cookie bằng header **Set-Cookie**; browser lưu cookie và tự động gửi lại trong header **Cookie** ở các request phù hợp.

Một session cookie nên có các thuộc tính:

- **HttpOnly**: JavaScript phía client không đọc được cookie, giảm rủi ro khi có XSS.
- **SameSite=Lax**: cookie được gửi trong điều hướng cùng site và một số top-level navigation an toàn, hạn chế CSRF cơ bản.
- **Path=/**: cookie áp dụng cho toàn bộ ứng dụng.
- **Max-Age** hoặc không đặt thời hạn: nếu không đặt thời hạn, cookie trở thành session cookie và thường hết hạn khi browser session kết thúc.

Trong implementation hiện tại, tên cookie được đặt theo port của HTTP app, ví dụ **session_2026**. Thiết kế này giúp tách biệt session giữa nhiều instance server đang chạy song song trên các cổng khác nhau. Cookie được lưu với **Path=/**, **HttpOnly**, **SameSite=Lax** và **Max-Age**; đây là mức cấu hình đủ tốt cho môi trường demo cục bộ bằng HTTP.

Ví dụ response sau login thành công:

Listing 1.3: Set session cookie sau khi xác thực

```
HTTP/1.1 200 OK
Set-Cookie: session_2026=<random-session-id>; Path=/; HttpOnly; SameSite=Lax;
    Max-Age=28800
Content-Type: application/json

{"ok": true}
```

Server lưu ánh xạ **session_id** -> **user/session state**. Các route bảo vệ kiểm tra cookie phiên; nếu thiếu hoặc hết hạn thì trả 401 **Unauthorized**. Thời gian sống của session và nonce đều được giới hạn để tránh việc token cũ bị tái sử dụng quá lâu.

Ví dụ request truy cập route bảo vệ bằng cookie:

Listing 1.4: Request đến route bảo vệ với session cookie

```
GET /api/self HTTP/1.1
Host: 127.0.0.1:2026
Cookie: session_2026=<random-session-id>
```

Ví dụ request logout và phản hồi xóa cookie:

Listing 1.5: Logout và hủy session cookie

```
POST /logout HTTP/1.1
```

```
Host: 127.0.0.1:2026
Cookie: session_2026=<random-session-id>

HTTP/1.1 200 OK
Set-Cookie: session_2026=; Path=/; HttpOnly; SameSite=Lax; Max-Age=0
Content-Type: application/json

{"status":"ok","message":"logged_out"}
```

1.3.4 Xử lý lỗi và phản hồi xác thực

Khi xác thực thất bại, server không chỉ trả mã lỗi mà còn cung cấp thông tin để client biết cách xác thực lại:

- Nếu request không mang cookie hợp lệ và cũng không có **Authorization** hợp lệ, server trả 401 Unauthorized kèm **WWW-Authenticate**.
- Với Basic authentication, challenge có thể yêu cầu client gửi lại username/password theo realm đã khai báo.
- Với Digest authentication, server phát **nonce** và **opaque**; nếu **nonce** cũ hết hạn, challenge có thể trả thêm cờ **stale=true** để yêu cầu client tạo lại digest mới.
- Nếu đăng nhập thành công nhưng session hết hạn ở request sau, client phải xác thực lại để nhận session mới.
- Nếu người dùng đã logout, session cũ bị vô hiệu hóa nên cookie cũ không còn giá trị ngay cả khi client vẫn cố gửi lại request cũ.

Ví dụ phản hồi khi chưa được xác thực:

Listing 1.6: Response 401 yêu cầu xác thực lại

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="AsynapRous"
Content-Type: application/json

{"status":"error","error":"authentication_required"}
```

1.3.5 Nhận xét về lựa chọn thiết kế

Giải pháp hiện tại có một số ưu điểm rõ ràng. Thứ nhất, nó bám sát đề bài khi hỗ trợ cả hai hướng là HTTP authentication và cookie-based session. Thứ hai, việc ưu tiên session cookie ở các request sau giúp giảm việc lặp lại credential nhạy cảm trên đường truyền.

Thứ ba, việc hỗ trợ thêm Digest cho phép hệ thống minh họa cơ chế challenge-response đầy đủ hơn so với chỉ dùng Basic.

Tuy nhiên, đây vẫn là implementation phục vụ mục đích học tập và demo cục bộ. Báo cáo nên nêu rõ rằng cookie hiện chưa dùng thuộc tính **Secure** vì hệ thống chạy trên HTTP thường thay vì HTTPS; do đó nếu triển khai thực tế thì cần bật HTTPS, bổ sung **Secure**, tăng cường chính sách hết hạn session và mở rộng cơ chế thu hồi session theo nhiều thiết bị hoặc toàn hệ thống nếu bài toán thực tế yêu cầu.

1.4 Client-server Bootstrap

Tracker là thành phần control plane chịu trách nhiệm bootstrap mạng peer. Tracker không chuyển tiếp live message; nhiệm vụ của nó là giúp peer biết ai đang online và có thể kết nối đến đâu.

1.4.1 Peer registration

Khi sampleapp khởi động PeerManager, peer gửi request đăng ký đến tracker với các thông tin:

- **peer_id**: định danh logic của peer;
- **host**: địa chỉ IP hoặc hostname mà peer lắng nghe;
- **port**: TCP listen port của PeerManager;
- **capabilities**: các tính năng hỗ trợ như chat, broadcast hoặc channel;
- **last_seen**: thời điểm tracker ghi nhận peer còn sống.

Tracker lưu danh sách active peers trong bộ nhớ hoặc file trạng thái. Mỗi peer record có TTL để tránh giữ peer đã chết mãi mãi.

1.4.2 Heartbeat

Peer gửi heartbeat định kỳ để gia hạn trạng thái online. Nếu tracker không nhận heartbeat trong một khoảng thời gian cấu hình trước, tracker đánh dấu peer là expired và loại khỏi kết quả discovery.

Heartbeat không cần chứa message chat. Nó chỉ là tín hiệu control plane cho biết peer vẫn đang lắng nghe và sẵn sàng kết nối.

1.4.3 Discovery

Khi peer A muốn kết nối peer B, sampleapp gọi tracker để lấy danh sách peer hoặc tra cứu theo `peer_id`. Tracker trả về endpoint của peer B, ví dụ `127.0.0.1:9002`. Sau đó peer A mở kết nối TCP trực tiếp đến peer B.

Luồng bootstrap:

1. Peer A và Peer B register lên tracker.
2. Hai peer gửi heartbeat định kỳ.
3. Peer A request discovery từ tracker.
4. Tracker trả endpoint của Peer B.
5. Peer A thiết lập TCP P2P đến Peer B.
6. Tin nhắn live đi trực tiếp qua kết nối P2P.

1.5 P2P Protocol Design

Đây là phần trọng tâm của hệ thống. Protocol P2P phải giải quyết ba vấn đề: đóng gói message trên TCP byte stream, xác định loại message, và quản lý trạng thái kết nối giữa hai peer.

1.5.1 Frame format

Mỗi message được đóng gói thành frame:

[4-byte length][JSON payload]

Phần length là số nguyên unsigned 32-bit theo network byte order, biểu diễn độ dài của JSON payload tính bằng byte. Payload là JSON UTF-8.

Listing 1.7: Ví dụ frame payload JSON

```
{
  "type": "chat",
  "id": "msg-001",
  "from": "peerA",
  "to": "peerB",
  "timestamp": 1733000000,
  "body": "Hello from peerA"
}
```

Length prefix giúp receiver biết khi nào đã nhận đủ một message hoàn chỉnh dù TCP chia nhỏ dữ liệu thành nhiều lần `recv()`.

1.5.2 Message types

Protocol định nghĩa các message type sau:

Type	Ý nghĩa
hello	Message đầu tiên sau khi kết nối TCP được mở. Chứa <code>peer_id</code> , version protocol và capabilities.
chat	Tin nhắn riêng từ một peer đến một peer khác.
broadcast	Tin nhắn gửi đến nhiều peer đang kết nối hoặc các thành viên cùng channel.
ack	Xác nhận đã nhận hoặc đã xử lý một message có id.
ping	Kiểm tra kết nối còn sống và đo độ trễ.
pong	Phản hồi cho ping.
goodbye	Thông báo đóng kết nối có chủ đích.
error	Báo lỗi protocol, ví dụ JSON sai schema, type không hỗ trợ hoặc handshake không hợp lệ.

1.5.3 Handshake

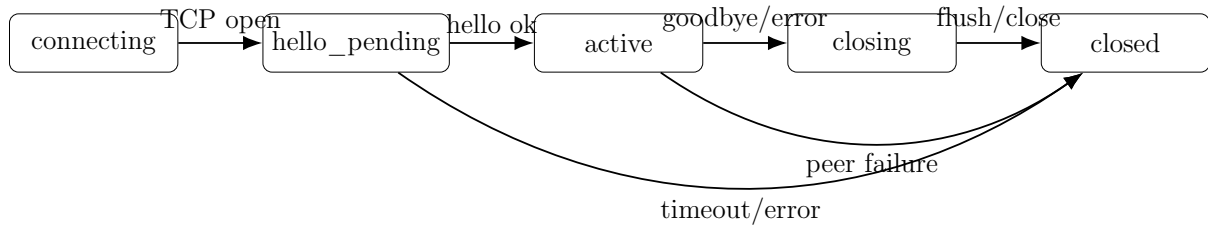
Sau khi mở TCP connection, peer chủ động gửi `hello`. Peer nhận kiểm tra version, peer id và capabilities. Nếu hợp lệ, peer nhận trả `hello` hoặc `ack`; sau đó kết nối chuyển sang trạng thái active.

Nếu quá thời gian mà không nhận được `hello`, PeerManager đóng connection để tránh giữ socket rác.

1.5.4 Connection state machine

Mỗi connection được quản lý bằng state machine:

- **connecting:** socket đang mở hoặc vừa accept, chưa hoàn tất handshake.
- **hello_pending:** đã gửi hoặc đang chờ `hello`.
- **active:** handshake thành công, có thể gửi `chat`, `broadcast`, `ping`.
- **closing:** đã gửi hoặc nhận `goodbye`, đang flush outbound queue.
- **closed:** socket đã đóng, connection bị loại khỏi selector và peer table.



Hình 1.2: State machine của một kết nối P2P

1.5.5 Reliability trong phạm vi ứng dụng

TCP đã đảm bảo byte stream tin cậy, nhưng protocol vẫn cần **ack** ở tầng ứng dụng để xác nhận message đã được parse và xử lý. Mỗi **chat** hoặc **broadcast** có **id**; receiver gửi **ack** kèm **id**. Sender có thể log message ở trạng thái pending, delivered hoặc failed.

1.6 Channel & DHT Extension

Phần này là mở rộng nâng cao, dùng để thể hiện distributed routing và synchronization. Thay vì để tracker quản lý toàn bộ channel tập trung, hệ thống có thể dùng ý tưởng Distributed Hash Table (DHT) để xác định peer chịu trách nhiệm cho từng channel.

1.6.1 Hash channel name thành key

Mỗi channel name được hash thành một key trong không gian định danh vòng, ví dụ:

$$\text{key} = \text{hash}(\text{"network-lab"}) \bmod 2^m$$

Mỗi peer cũng có một key định danh. Owner của channel là successor của channel key trên vòng DHT, tức peer đầu tiên có id lớn hơn hoặc bằng key theo chiều vòng.

1.6.2 Route qua ring

Khi peer muốn join hoặc gửi message vào channel, peer không cần biết owner trực tiếp. Request có thể được route qua ring:

1. peer tính key từ channel name;
2. peer chuyển request đến neighbor gần key hơn;
3. request tiếp tục đi qua ring cho đến successor;
4. successor trở thành owner xử lý membership và fan-out.

1.6.3 Owner fan-out đến members

Owner lưu danh sách members của channel. Khi nhận message cho channel, owner fan-out đến các member đang active. Nếu một member unreachable, owner đánh dấu failure và có thể retry hoặc loại khỏi membership sau TTL.

Cách này cho thấy sự khác nhau giữa tracker bootstrap và distributed routing. Tracker chỉ giúp peer vào mạng ban đầu; sau đó việc định tuyến channel có thể được phân tán qua ring.

1.6.4 Synchronization

Vì owner có thể rời mạng, DHT extension cần cơ chế synchronization:

- replicate metadata channel sang successor kế tiếp;
- heartbeat giữa các node liền kề trong ring;
- transfer ownership khi phát hiện owner cũ failed;
- version hoặc timestamp cho membership update để tránh ghi đè dữ liệu cũ.

1.7 Testing & Failure Handling

Kiểm thử tập trung vào protocol P2P, cơ chế non-blocking I/O và hành vi khi có lỗi. Các test đã passed chứng minh hệ thống xử lý được cả luồng bình thường và các tình huống biên.

1.7.1 Test cases

Test case	Kết quả mong đợi
Send before connect	PeerManager từ chối gửi, trả lỗi rõ ràng hoặc queue theo chính sách cấu hình; không crash.
Malformed message	Receiver phát hiện JSON sai hoặc thiếu field bắt buộc, gửi error và có thể đóng connection.
Handshake timeout	Connection ở hello_pending quá thời hạn sẽ bị đóng và cleanup khỏi selector.
Peer failure cleanup	Khi peer đóng bất ngờ hoặc socket lỗi, PeerManager loại connection khỏi peer table và giải phóng buffer/queue.

Duplicate connect	Nếu hai peer mở kết nối trùng nhau, hệ thống giữ một connection hợp lệ theo tie-break rule, ví dụ so sánh <code>peer_id</code> .
Broadcast partial failure	Broadcast vẫn gửi đến các peer còn sống; peer lỗi được ghi nhận riêng, không làm hỏng toàn bộ thao tác.

1.7.2 Failure handling

Malformed frame: nếu length prefix quá lớn, âm theo parse lỗi, hoặc payload không phải JSON hợp lệ, receiver gửi **error** rồi đóng kết nối để bảo vệ event loop.

Partial send failure: nếu socket lỗi khi outbound queue còn dữ liệu, message liên quan được đánh dấu failed. Các message đã ack trước đó vẫn giữ trạng thái delivered.

Handshake failure: nếu peer không gửi **hello**, gửi sai version hoặc trùng `peer_id` bất thường, connection không được chuyển sang active.

Heartbeat và ping/pong: trong data plane, **ping/pong** giúp phát hiện kết nối chết nhưng chưa đóng rõ ràng. Nếu quá nhiều lần ping timeout, PeerManager cleanup socket và thông báo lên sampleapp.

1.7.3 Kết luận kiểm thử

Các test passed cho thấy thiết kế tách control plane/data plane hoạt động đúng: tracker hỗ trợ bootstrap và discovery, còn live message đi qua TCP P2P. Non-blocking event loop giúp PeerManager xử lý nhiều socket mà vẫn kiểm soát được partial frame, outbound queue và cleanup khi lỗi.